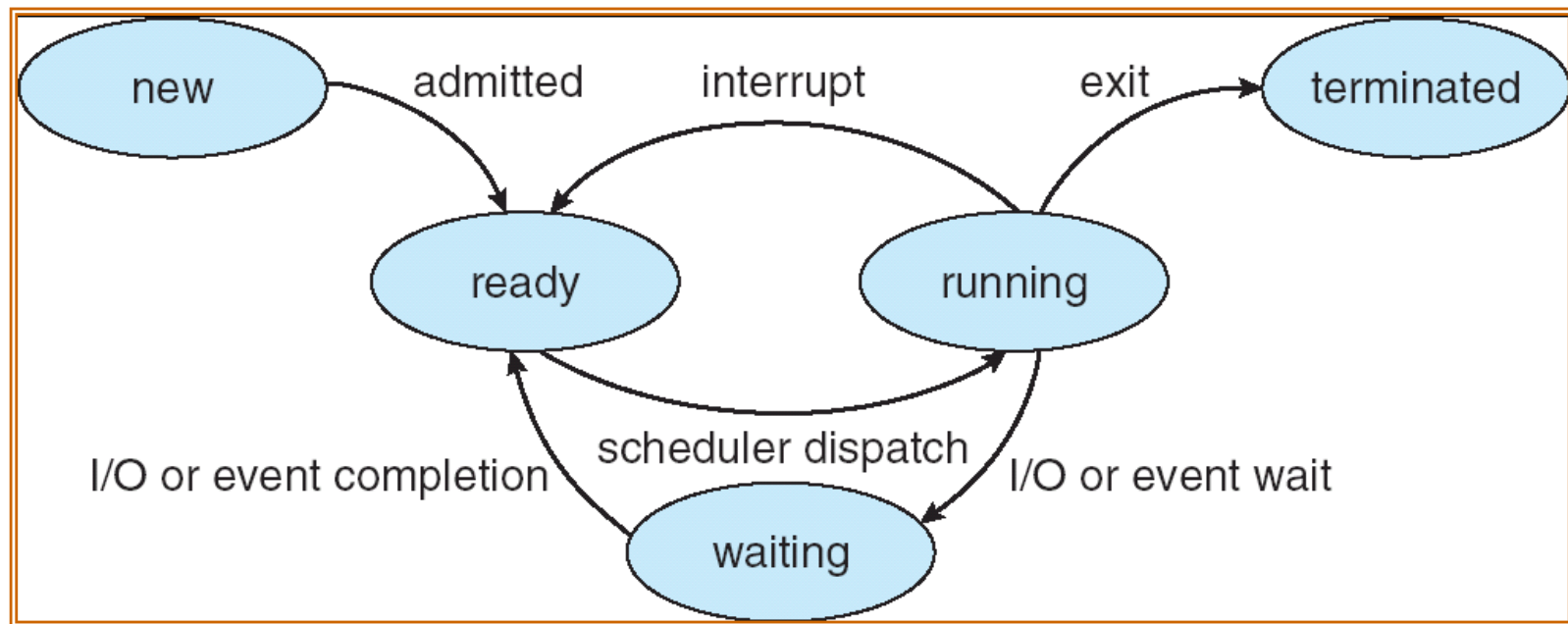


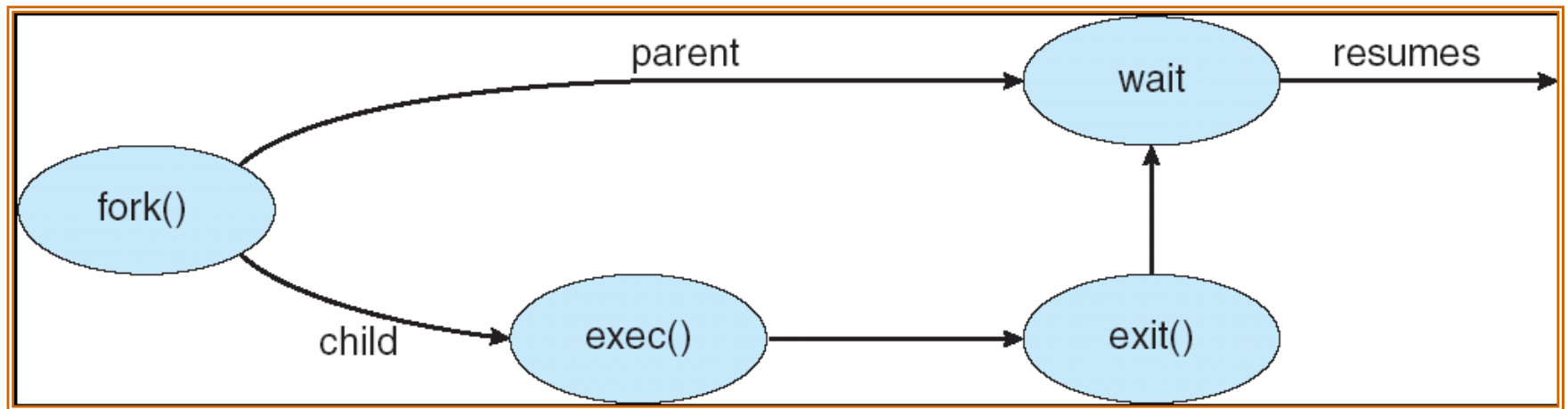
Chapter 13: Systems Programming (Process Management)

Instructor: Dr. Md Mahfuzur Rahman
Department of Computer Science, GSU

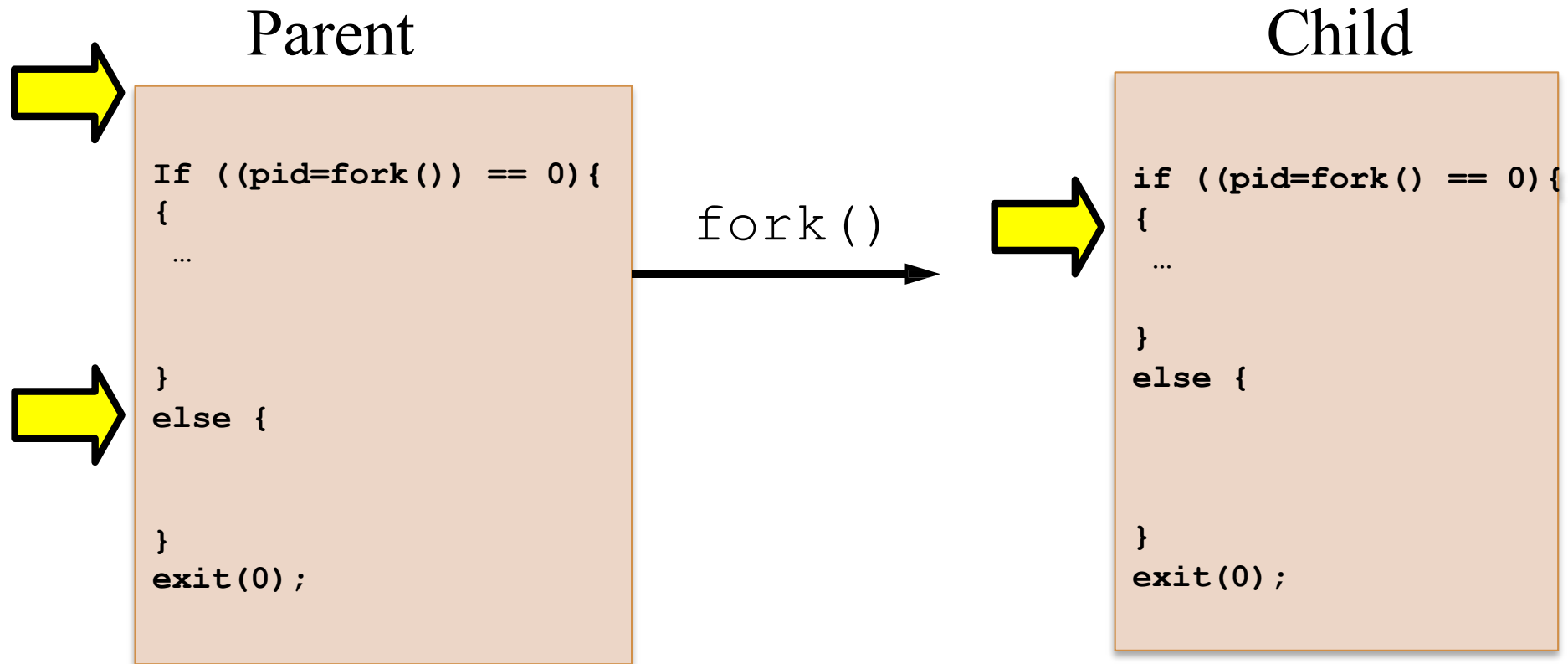
Process States



Process Creation & Termination



What's a Fork ()



- Child is an exact copy of the parent process.
- They have their own memory space.

How to use a Fork () ?

`pid_t fork(void)`

- Related header files
 - `#include <sys/types.h>`
 - `#include <unistd.h>`
- Call once but return twice
 - 0 for the child process
 - Child *pid* for the parent
- Cannot predict which process runs first
 - Process scheduling
- Run the same code concurrently, but have completely separate stack and data space

Process creation

```
#include <stdio.h>
#include <unistd.h>
int main() {
    printf("Before fork()\n");

    pid_t pid = fork();
    if (pid == 0) {
        // Code executed by the child process
        printf("Child process: PID = %d\n", getpid());
    }
    else if (pid > 0) {
        // Code executed by the parent process
        printf("Parent PID = %d, Child PID = %d\n", getpid(), pid);
    }
    else {
        // fork() failed
        perror("fork failed");
    }
    printf("This line is executed by both parent and child processes\n");
    return 0;
}
```

Observing parallelism

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    printf("Start of the program (PID = %d)\n", getpid());
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        for (int i = 0; i < 5; i++) {
            printf("Child process (PID = %d): i = %d\n", getpid(), i);
            sleep(1);
        }
    } else if (pid > 0) {
        // Parent process
        for (int i = 0; i < 5; i++) {
            printf("Parent process (PID = %d): i = %d\n", getpid(), i);
            sleep(1);
        }
    } else {
        perror("fork failed");
    }
    return 0;
}
```

More about fork()

- Reasons for fork to fail
 - Too many processes in the system
 - The total number of processes for the real *uid* (User ID) exceeds the limit
 - ✦ CHILD_MAX
- Usages of fork
 - Duplicate a process to run different sections of code
 - ✦ Network servers
 - Want to run a different program
 - ✦ fork+exec



Differentiating a Process: exec

- Use the absolute or relative name
 - `int execl(const char* path, const char* arg0, ..., const char* argn, NULL)`
 - `int execv(const char* path, const char* argv[ ])`
- Only the code that the process is executing changes

Example Using `exec1()`

```
#include<unistd.h>
#include<stdio.h>
int main()
{
    printf ("I'm process %d and I'm about to exec a ls -l\n", getpid());
    exec1 ("/bin/ls", "ls", "-l", NULL); /* Execute ls */
    printf ("This line should never be executed\n");
    return 0;
}
```

\$myexec

I'm process 61713 and I'm about to exec a ls -l

total 6

```
-rwxr-xr-x  1 mrahman21  staff  33736 Nov 18 00:08 a.out
-rwx-----@ 1 mrahman21  staff   8572 Sep  4 18:38 array
-rw-r--r--  1 mrahman21  staff   408 Oct 18 16:59 array_modify.c
-rwxr-xr-x  1 mrahman21  staff 33592 Oct 22 21:40 array_ptr
-rw-r--r--  1 mrahman21  staff   473 Oct 22 21:40 array_ptr_increment.c
-rw-r--r--  1 mrahman21  staff   422 Oct  8 21:32 array_side_effect.c
```



A simple example using fork()

```
#include<stdio.h>
#include<unistd.h>
int main () {
    int pid;
    printf ("I'm the original process with PID %d and PPID %d.\n",getpid (), getpid ());
    pid = fork (); /* Duplicate. Child & parent continue from here */
    if (pid != 0) /* pid is non-zero, so I must be the parent */
    {
        printf ("I'm the parent process with PID %d and PPID %d.\n",getpid (), getpid ());
        printf ("My child's PID is %d\n", pid);
    }
    else /* pid is zero, so I must be the child */
    {
        printf ("I'm the child process with PID %d and PPID %d.\n",getpid (), getpid ());
    }
    printf ("PID %d terminates.\n", getpid () ); /* Both processes execute this */

    return 0;
}
```

- getpid() returns the PID(int) of current process.
- getppid() returns the PID(int) of parent process



A simple example using fork()

\$ myfork

I'm the **original process** with **PID 639** and **PPID 416**.

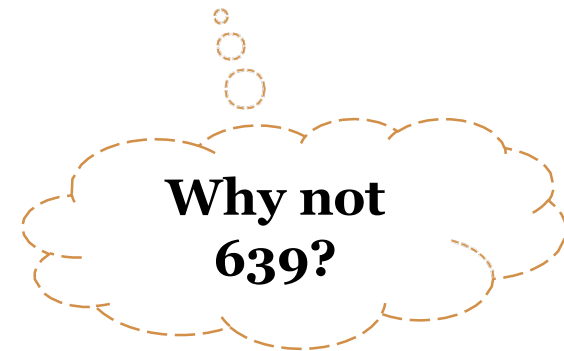
I'm the **parent process** with **PID 639** and **PPID 416**.

My child's PID is 640

PID 639 terminates.

I'm the **child process** with **PID 640** and **PPID 1**.

PID 640 terminates.



Orphan Processes

- If a parent process terminates before its child terminates, the child process is automatically adopted by the *init* process
- The following program shows this.

Example Program of Orphan

The parent process finishes earlier than its child, making the child an “orphan” process.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
int main () {
    int pid;
    printf ("I'm the original process with PID %d and PPID %d.\n", getpid(), getppid());
    pid = fork (); /* Duplicate. Child & parent continue from here */
    if (pid != 0) /* Branch based on return value from fork () */ {
        /* pid is non-zero, so I must be the parent */
        printf ("I'm the parent process with PID %d and PPID %d.\n",getpid (), getppid ());
        printf ("My child's PID is %d\n", pid);
    }
    else {
        /* pid is zero, so I must be the child */
        sleep(5); /* Make sure that the parent terminates first */
        printf ("I'm the child process with PID %d and PPID %d.\n",getpid (), getppid ());
    }
    printf ("PID %d terminates.\n", getpid() ); /* Both processes execute this */
    return 0;
}
```

Example Program of Orphan

\$ orphan

I'm the **original** process with PID **680** and PPID **416**.

I'm the **parent** process with PID **680** and PPID **416**.

My child's PID is **681**

PID 680 terminates.

\$ I'm the **child** process with PID **681** and PPID **1**.

PID 681 terminates.

Zombie Processes

- A process cannot leave the system until parent process accepts its termination code (returned by `exit`)
- If parent process is dead; *init* adopts process and accepts code **Orphan process**
- If the parent process is alive but is unwilling to accept the child's termination code
 - because it never executes `wait()` **Zombie process**
 - the child process will remain a zombie process.

Zombie Example

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
int main () {
    pid_t pid;
    pid = fork (); /* Duplicate */
    /* Branch based on return value from fork () */
    if (pid != 0) {
        /* Never terminate, never execute a wait () */
        while (1)
            sleep (1000);
    }
    else {
        exit (42); /* Exit with a silly number */
    }
    return 0;
}
```

The parent process continues to run but it is not waiting/willing to receive the termination code from its child, eventually making the child a “zombie” process.

Zombie Example

\$ **zombie &**

[1] 684

\$ **ps**

684 p4 S 0:00 zombie

685 p4 Z 0:00 (zombie <zombie>)

686 p4 R 0:00 ps

\$ **kill 684**

[1]+ Terminated zombie

\$ **ps**

688 p4 R 0:00 ps

\$

Waiting for a Child: wait()

- If a process executes a wait(),
 - And one or more children are already zombies,
 - Then wait() returns immediately with the status of one of the zombies



Example Program using wait()

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
int main () {
    int pid, status, childPid;
    printf ("I'm the parent process and my PID is %d\n", getpid());
    pid = fork (); /* Duplicate */
    if (pid != 0) /* Branch based on return value from fork () */
    {
        printf ("I'm the parent process with PID %d and PPID %d\n",getpid (), getppid ());
        /* Wait for a child to terminate. */
        childPid = wait (&status);
        printf("A child with PID %d terminated with exit code %d\n", childPid, status >> 8);
    }
    else {
        printf ("I'm the child process with PID %d and PPID %d\n", getpid (), getppid ());
        exit (42); /* Exit with a silly number */
    }
    printf ("PID %d terminates\n", getpid ());
    return 0;
}
```

Example Program using wait()

\$ myprog

I'm the parent process and my PID is 66743

I'm the parent process with PID 66743 and PPID 10505

I'm the child process with PID 66744 and PPID 66743

A child with PID 66744 terminated with exit code 42

PID 66743 terminates